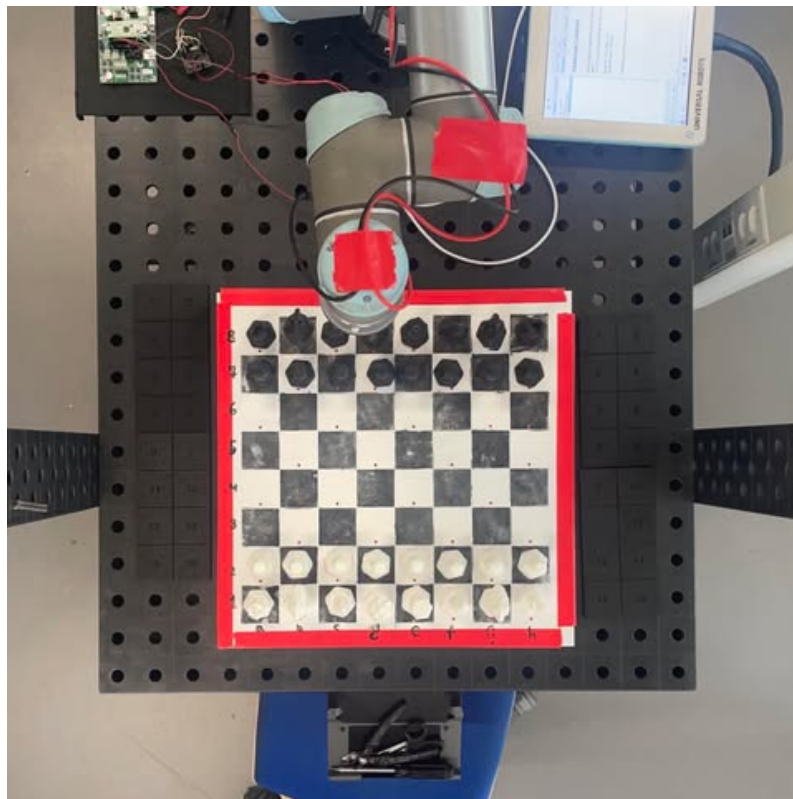


UNIVERSITY OF SOUTHERN DENMARK

Autonom Robot

UR5 - GRIPPER - SKAKINTELLIGENS



Robotteknologi 2. Semester

27. Maj 2025

Gruppe 3

August Maximillian Rosford Tranberg (06/09/04) (autra24)

Emil Gramstrup (25/01/04) (egram24)

Magnus Meldgård (16/09/98) (mmeld24)

Rune Kildahl Frederiksen (08/02/03) (rufre23)

Resumé

This project presents the development of an autonomous robotic system designed to play chess against a human opponent. The system integrates a UR5 robotic arm equipped with a custom-designed gripper and an electronic chessboard, creating an interactive and tactile chess-playing experience.

The robotic arm, controlled via the Robot Operating System (ROS), utilizes precise Cartesian-space movements to manipulate chess pieces on the board. The gripper, driven by a DC motor and monitored through current sensing, ensures accurate and reliable piece handling. The electronic chessboard, featuring reed relays and LEDs, provides real-time feedback and visual guidance to the player, enhancing the user experience.

The system's software architecture combines ROS, C++, and the Stockfish chess engine to facilitate intelligent move calculation, real-time communication, and precise motion planning. The project successfully demonstrates the feasibility of creating a user-friendly and functional chess-playing robot, offering a unique blend of physical interaction and advanced robotics.

Future developments could include enhancing the gripper's precision, expanding the software to handle all special chess moves, and improving the user interface for a more immersive experience. This project serves as a testament to the potential of robotic systems in educational and recreational applications.

Titelblad

Projekttitel	RB-PRO2 - UR Gripper (Semester Project in Autonomous Robots)
Undervisningssted	Syddansk Universitet, Det Tekniske Fakultet
Uddannelse	Bachelor i robotteknologi
Antal bilag	6
Projektperiode	3. februar 2025 - 27. maj 2025
Projektvejledere	Anders Stengaard Sørensen (anss@mmmi.sdu.dk) Simon Faarvang Mathiesen (simat@mmmi.sdu.dk)
Projektgruppenummer	3
Projektdeltagere (eksamensnummer)	August Maximillian Rosford Tranberg (215661973) Emil Gramstrup (4139374) Magnus Meldgård (215661926) Rune Kildahl Frederiksen (4138696)
GitHub link	https://github.com/Gubi0609/Semesterprojekt-i-autonome-robotter/tree/main
Demovideo af system	https://www.youtube.com/shorts/Ke06ArgRxZI
Ansvarsområder:	
August T.	Stockfish, Skakbræt (3D design)
Emil G.	Gripper (3D design support), Skakbrikker
Magnus M.	UR5 (ROS og Kinematik), Gripper (3D design) og Skakbrikker
Rune F.	Gripper (software og elektronik), Skakbræt (software og elektronik), Kommunikation mellem enheder

Indhold

1	Indledning	3
2	Projektbeskrivelse	4
3	Metode	5
4	Teori	6
4.1	DC-Motor og PWM (Pulse Width Modulation)	6
4.2	Differentialforstærker (Op Amp)	7
4.3	Shuntmodstand	8
4.4	Multiplexing	8
4.5	ROS (Robot Operating System)	9
4.6	UR5	9
4.7	Stockfish	10
5	Analyse	11
5.1	Skakbræt	11
5.2	Gripper og skakbrikker	13
5.2.1	Testresultater fra gripper og skakbrik sammenspil	14
5.2.2	Gripper Elektronik	15
5.2.3	Gripper Software	18
5.2.4	Testresultater fra gripper	19
5.3	Pico-UR5-Ubuntu Kommunikation	20
5.4	UR5 og ROS	21
5.4.1	Testresultater fra UR5	26
6	Diskussion	27
6.1	Versionskontrol – Git	27
6.2	Gripper og Skakbræt	27
6.3	Op amp	28

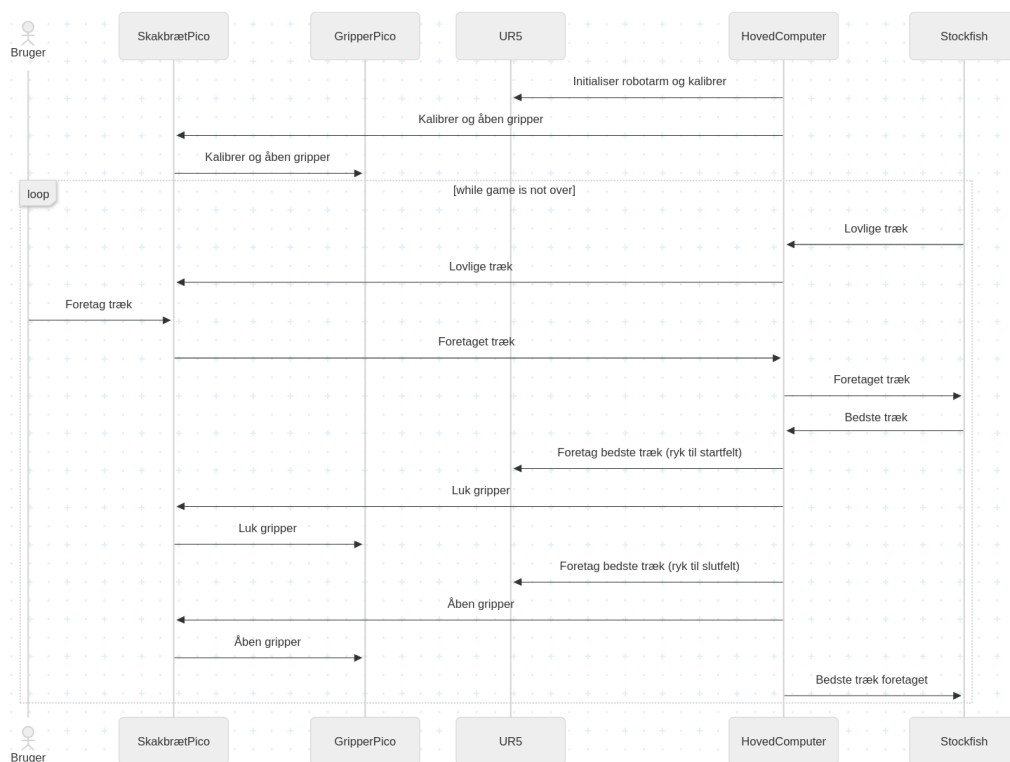
6.4	UR5 og ROS	29
6.5	Skakbræt eller Computer Vision	30
6.6	Pico-UR5-Ubuntu Kommunikation	30
6.7	Specielle skaktræk	31
7	Konklusion	32
8	Perspektivering	33
9	Litteratur	34
10	Appendix	35

1 Indledning

I en stadig mere digitaliseret verden udfordres de fysiske rammer for læring og fællesskab. Med dette projekt ønsker vi at genoplive det taktile og sociale aspekt af spil og læring – uden at afvise digitaliseringen som helhed. Vi har derfor udviklet en gripper og et robotsystem, der gør det muligt at spille og lære skak på et fysisk, elektronisk skakbræt med en UR5-robotarm som modstander.

Systemet er designet til at guide brugeren gennem spillet uden behov for konstant skærminteraktion. I stedet modtager brugeren visuel feedback via skakbrættets indbyggede LED'er, som angiver mulige træk og hvornår disse må udføres. Dette gør spiloplevelsen mere intuitiv og nærværende – særligt for begyndere og brugere uden teknisk baggrund.

Af hensyn til både brugervenlighed og systemets kompleksitet har vi valgt ikke at benytte computer vision. I stedet anvendes Reed-relæer i brættet til registrering af brikernes position. Denne løsning er mere tilbagemeldende, hvilket muliggør oftere opdatering af spildata og en mere direkte interaktion mellem spiller og system.



Figur 1: Sekvens diagram for interaktion med systemet.

2 Projektbeskrivelse

Formålet med projektet er at designe og fremstille en gripper til en UR5 robot, som kan lukkes/åbnes, og som kan læse data, der skal overføres til og visualiseres på en central computer. Der skal bruges en mikrocontroller til at styre gripperens aktuatorer og sensorer, og gripperen skal være tilrettelagt en specifik use-case.

Gripperen skal opfylde følgende krav:

- 24 V ekstern forsyning .
- Maksimum strøm-brug: 600 mA.
- Må ikke gøre brug af digital I/O fra UR5 robotten.

Projektets andre dele skal opfylde følgende krav:

- Det centrale program til at styre robot og gripper skal være skrevet i C++.
- Præstationen af gripper skal testes og dokumenteres.
- Brugen af administrerende værktøjer/strategier skal dokumenteres.

Gripperen i dette projekt er designet til at fungere i sammenspil med et elektronisk skakbræt. Formålet med skakbrættet og gripperen, er at give nye spillere mulighed for at øve sig i mod en lærende og passende modstander på et fysisk skakbræt. Systemet kan også installeres i sociale rum, hvor mennesker af alle aldre vil have mulighed for at spille et parti skak mod en computer på et fysisk skakbræt.

3 Metode

Til dette projekt har vi taget udgangspunkt i metoder og teknologier, som vi har udvalgt nøje ud fra undersøgelse af de forskellige muligheder. Projektet kombinerer programmering, mekanisk design og elektronik for at realisere en robot, der kan spille skak på et fysisk skakbræt med brug af en UR5-robotarm.

Vi har anvendt versionskontrol med Git til at holde styr på udviklingsprocessen og samarbejdet i gruppen. Dette har muliggjort struktureret kodning og tilbageblik på tidligere versioner af både software og dokumentation. Versionsstyringen har desuden, sammen med kanbanboard, gjort det lettere at organisere koden og uddelegere opgaver.

Som input til robotten benyttes et fysisk skakbræt udstyret med reed-relæer, der registrerer brikernes placering via magneter. Denne tilgang er valgt som alternativ til computer vision, da det gør det muligt samtidigt at lyse felter op, som spilleren kan rykke til. Det muliggør samtidigt en registrering af brikernes position i realtid, hvilket er centralt for systemets funktion.

Griperen, som robotarmen bruger til at flytte skakbrikkerne, er specialdesignet til formålet og udformet med fokus på præcision, skånsomhed og enkel mekanisk opbygning. Designet er iterativt udviklet og senere raffineret med 3D-printede komponenter.

Under udviklingen har vi haft særlige fokuspunkter, såsom nøjagtighed i positionering, stabilitet i brikkeflyt, enkelthed af brugerflade, så alle burde kunne gøre brug af systemet. Disse parametre er løbende blevet evalueret gennem praktiske tests og justeringer af både hardware og software.

Softwarearkitekturen er bygget op omkring ROS (Robot Operating System), som har dannet grundlag for kommunikation mellem systemets komponenter. ROS har fungeret som mellemlid mellem URDF-beskrivelsen af robotten og URDriver, hvilket har muliggjort en struktureret kommunikation og kontrol af robotarmen. Samtidig har ROS' indbyggede værktøjer understøttet simulering, debugging og integration med UR5-robotarmen.

Projektets beslutninger er truffet ud fra en målsætning om at opnå et pålideligt, brugervenligt og funktionelt skakspil mellem en fysisk robot og en menneskelig modspiller. Undervejs er der blevet foretaget nødvendige afvejninger mellem kompleksitet, robusthed og implementeringstid for at opnå det bedst mulige resultat inden for projektets rammer.

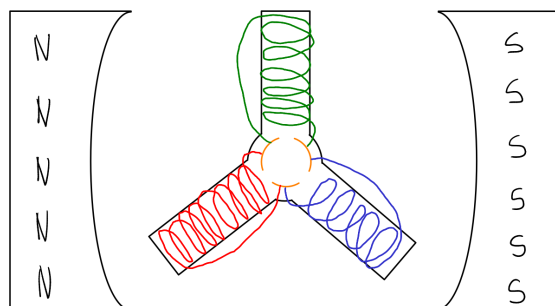
4 Teori

Dette afsnit omhandler den relevante teori for at forstå projektet.

4.1 DC-Motor og PWM (Pulse Width Modulation)

PWM bruges i et elektrisk kredsløb til at styre den gennemsnitlige forsyningsspænding. Dette gøres ved først at sætte forsyningsspændingen til 100% i et stykke tid og dernæst til 0% i et tilsvarende stykke tid. Produktet af forholdet mellem de to tidsintervaller og den fulde forsyningsspænding er den gennemsnitlige forsyningsspænding. Ved at variere forskellen mellem de to tidsintervaller kan man styre den gennemsnitlige styrke. Dette kan bruges i samspil med DC-motorer til at styre hastigheden, hvormed de drejer. [1]

Projektet anvender en DC-motor som drivkraft til gripperen på robotarmen. En DC-motor består grundlæggende af en stator (den stationære del) og en rotor (den roterende del). Statoren udgøres af to magnetpoler placeret omkring rotoren. Rotoren består af mindst tre ben med viklinger af elektrisk ledende tråd, som er forbundet til en kommutator. Kommutatoren er opdelt svarende til antallet af ben og gør det muligt at opnå en løbende ombytning af strømretningen, hvilket sikrer en fortsat rotation i samme retning. Rotoren er monteret på en aksel, hvilket tillader overførsel af den roterende bevægelse. Når der sendes strøm gennem en vikling, dannes et magnetfelt, som bliver påvirket af statorens magnetfelt og derved frembringer en drejende kraft. Desuden bliver en DC-motors indre modstand højere, desto hurtigere den drejer. På den måde når den et punkt, hvor omdrejningshastigheden ikke kan forøges mere. [2]

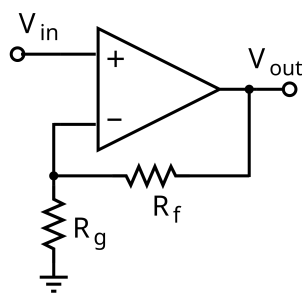


Figur 2: En simpel DC Motor

4.2 Differentialforstærker (Op Amp)

I projektet vil vi gerne måle strømmen gennem gripperens DC-motor. Da dette gøres over en shuntmodstand, vil den resulterende spænding og derved strøm være meget lav. For at vi kan måle det med Raspberry Pi Picos ADC-indgang, bruger vi en Op Amp med negativ feedback til at øge signalets styrke.

En Op Amp er en spændingsforstærker med differential indgang og én udgang. Udover dette har en Op Amp i *open loop* en meget høj *gain*, men da vi gør brug af *closed loop* dykker vi ikke dybere ned i det. Ved at gøre brug af negativ feedback kan man styre Op Ampens udgangsspænding via eksterne komponenter i stedet for dens egne karakteristika. En Op Amp har desuden en positiv og negativ forsyningsspænding, der udgør dens øvre og nedre grænse for outputspændingen.



Figur 3: Ikke-inverterende negativ feedback på en Op Amp [9]

Vi bruger i projektet ikke-inverterende negativ feedback til at forstærke den målte spænding over en shunt modstand, så vi kan måle strømmen gennem motoren. Når man gør brug af denne feedbackmetode, kan forstærkelsen bestemmes af to modstande og udregnes ved brug af denne formel:

$$V_{out} = V_{in} \left(1 + \frac{R_f}{R_g} \right)$$

[9]

4.3 Shuntmodstand

En shunt er en modstand, der er designet eller valgt, så den tillader en vej med lav modstand, som strømmen kan løbe i et kredsløb. En shunt kan bruges på mange måder i et kredsløb. Vi bruger den til at måle strømmen, der løber i DC-motor kredsløbet, og derved strømmen gennem DC-motoren. En shunt-modstand med meget lav, men kendt modstand, placeres i serie med DC-motoren. Modstanden vælges, så den ikke påvirker strømmen i kredsløbet med en bemærkelsesværdig effekt. Shunt-modstanden placeres i parallel med et volt-meter, så man kan måle spændingen over shunt-modstanden. Da man nu kender resistansen og spændingen over shunt-modstanden, kan man bruge Ohms lov

$$I = \frac{U}{R}$$

til at udregne strømmen gennem shunt-modstanden og derved DC-motoren. [10] Vi har valgt en shunt-modstand med en resistans på 0.5Ω , og vores DC-motor er en HG37-150-AB-00, med en stillestående resistans på 66.5Ω . Altså er vores shunt-modstand på kun 0.75% af DC-motorens resistans.

Som volt-meter bruger vi en Raspberry Pi Pico og en differentialforstærker, så den målte spænding er høj nok til at kunne måles effektivt af Picoens ADC-port. ADC-porten måler i 12 bits opløsning. Altså har vi en opløsning fra 0 til 4096, hvor 0 er 0 V og 4096 er 3.3 V.

4.4 Multiplexing

Multiplexing er en metode til at sende eller modtage flere signaler over ét eller flere kabler. Formålet er at udnytte et fysisk medie såsom ledninger til flere signaler, i det tilfælde at man ikke kan opdele signalet i flere ledninger. Vi gør brug af *Time-division multiplexing* (TDM), da vi har en matrice af ledninger, som vi itererer igennem og tjekker efter en forbindelse. På den måde kan vi udnytte Picoens GPIO-pins mere effektivt, da vi ellers ikke havde nok, hvis vi skal tjekke 64 reed-relæer og styre 64 LED'er. [11]

4.5 ROS (Robot Operating System)

Robot Operating System (ROS) er et open-source middleware-rammeverk udviklet til at lette udviklingen af software til robotteknologi. ROS fungerer som et operativsystem i et netværks-distribueret miljø og muliggør kommunikation mellem softwarekomponenter kaldet “noder”. Det bruges bredt i både forskning og industri på grund af dets fleksibilitet, modulopbygning og store community-understøttelse.

Et centralt element i ROS er dets kommunikationsarkitektur, som bygger på tre hovedprincipper: topics, services og actions. Topics bruges til asynkron dataudveksling mellem noder via en publisher-subscriber-model. Dette er velegnet til f.eks. sensor- og tilstandsinformation. Services bruges til synkron kommunikation, hvor en node kan anmode en anden om at udføre en bestemt handling og vente på et svar. Actions bruges til længerevarende processer, såsom robotbevægelse, hvor feedback og mulighed for afbrydelse er nødvendigt.

Til styring af robotarme som UR5 anvendes ROS ofte sammen med pakker som MoveIt, der håndterer bevægelsesplanlægning, inverse kinematik og kollisionsstjek. MoveIt arbejder sammen med UR5-driveren, som sørger for direkte kommunikation med robotcontrolleren via TCP/IP. Ved hjælp af MoveGroupInterface kan der defineres målpunkter i rummet, og ROS beregner herefter en gyldig bane for robotten.

ROS anvender desuden værktøjer som RViz til 3D-visualisering af robot, omgivelser og planlagte baner, samt tf til håndtering af koordinattransformationer mellem forskellige referenceframes i robotens kinematiske kæde.

I denne rapport danner ROS grundlaget for hele kommunikations- og styringslaget af UR5-robotten, og det muliggør integration med både softwarekomponenter (f.eks. skakmotoren Stockfish) og hardwareelementer (f.eks. griber og sensorer). [7]

4.6 UR5

Universal Robots UR5 er en industrielt anvendt, kollaborativ robotarm med seks frihedsgrader (6 DoF). Den er designet til at udføre præcise og fleksible bevægelser og anvendes ofte i automatisere-

ringsopgaver, forskning og udvikling, hvor menneske og robot arbejder i tæt samspil.

UR5 har en rækkevidde på cirka 850 mm og en maksimal nyttelast på 5 kg, hvilket gør den velegnet til opgaver som montage, materialehåndtering, inspektion og i dette tilfælde – skakspil. Robotten har en gentagelsesnøjagtighed på ± 0.1 mm, hvilket muliggør præcise placeringer og manipulation i kartesiske koordinater.

Til styring anvender UR5 et internt realtidsoperativsystem og modtager kommandoer via et Ethernet-interface. I ROS-miljøet etableres forbindelsen gennem Universal Robots' officielle ROS-driver, som giver adgang til robotarmens tilstand og tillader eksekvering af bevægelseskommandoer i realtid. Denne driver gør det muligt at integrere robotten med MoveIt til planlægning og simulering samt RViz til visualisering.

UR5's mekaniske struktur består af seks roterende led (revolute joints), hvilket gør det muligt at nå mange forskellige positioner og orienteringer i rummet. Det betyder også, at robotten er underlagt visse begrænsninger, såsom arbejdsområdets grænser og potentielle singulariteter, hvilket må tages i betragtning under bevægelsesplanlægning.

I dette projekt fungerer UR5 som den fysiske aktør, der modtager information om skaktræk og udfører dem i det fysiske miljø gennem præcist planlagte bevægelser. [5][6]

4.7 Stockfish

Vi har til projektet gjort brug af *Stockfish Chess Engine*. Stockfish bruges i projektet til at udregne lovlige træk for spilleren, og det bedste træk for robotten. For at udregne det bedste træk, gør Stockfish brug af en søgemetode, der hedder *Minimax* sammen med *pruning*. En skakposition kan evalueres ud fra et positivt eller negativt tal. Hvis tallet er positivt, er hvids position bedst og omvendt for sort. Stockfish evaluerer hver skakposition i dens søgen, og leder efter den evaluering, der er bedst for den givne spiller. Da dette hurtigt kan føre til et stort søgetræ, der tager lang tid at søge, bruger Stockfish samtidig brug af *pruning* til at afklippe de grene i søgetræet, som har en dårligere evaluering. [8]

Stockfish køres gennem terminalen, og vi kan derfor kommunikere med den gennem vores C++

program ved at bruge *pipes* til at omlægge output fra vores program til Stockfishs input, og Stockfishs output til vores program. På den måde kan vi lagre det bedste træk og de lovlige træk og gøre brug af det til at bevæge robotten eller styre lysene på vores skakbræt.

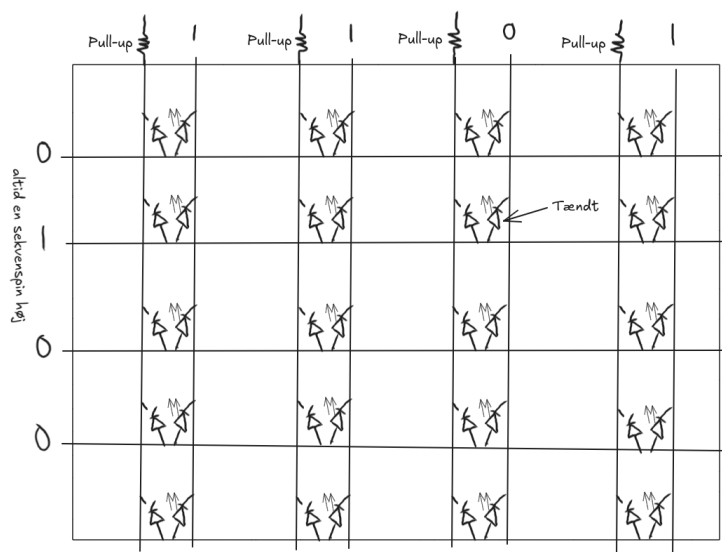
5 Analyse

5.1 Skakbræt

Skakbrættet er konstrueret med 64 reed-relæer til detektering af magnetiske felter på hvert felt samt 64 lysdioder, der anvendes til visuel oplysning af felterne. Hele systemet forsynes med en spænding på 3,3 V leveret af en Raspberry Pi Pico. Til registrering af skakbrikkernes position blev reed-relæer valgt, idet de nemt kan fastgøres under brættet, samtidig med at de effektivt indikerer tilstedeværelsen af en magnet på et givent felt.

For at identificere hvilke reed-relæer der aktiveres, anvendtes tidsmultiplexing (TDM). Her sendes skiftevis 3,3 V til hver af de otte sekventielle pins, hvorefter det kontrolleres, om de tilknyttede pull-up-resistorer på Pico'en registrerer et højt eller lavt signal. Denne tilgang muliggør aflæsning af samtlige 64 reed-relæer ved brug af blot 16 GPIO-pins.

Styringen af lysdioderne blev ligeledes baseret på multiplexing, idet alle dioder blev tilkoblet de samme otte sekvenspins. For at afgøre, om en LED skal aktiveres i en given sekvens, sættes den ene terminal til 0 V via en dedikeret GPIO-pin på Pico'en. Denne metode tillader styring af 64 lysdioder med kun otte ekstra pins. Dog medfører teknikken, at hver diode maksimalt kan være aktiv 1/8 af tiden, da kun ét sekvenspin kan være højt ad gangen.



Figur 4: En 1:4 model af brættet med illustration af hvordan vi tænder en LED

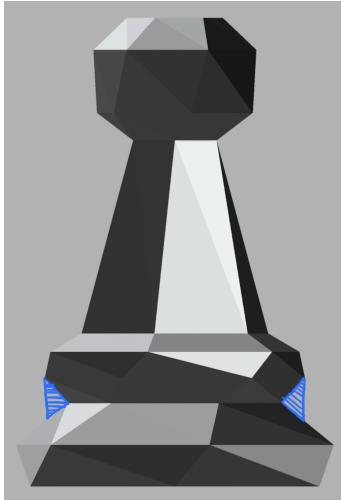
Det tager omkring 40 ms at scanne alle 64 Reed-relæer med Picoen. Der er ikke behov for at flytte hånden væk fra spillet eller lignende. Brættet ved, at der er flyttet en brik/magnet, når det registrerer en ændring på et felt. Det vil kun godtage trækket, hvis det er et lovligt træk. Lovlige træk bliver sendt som en vektorstreng fra hovedcomputeren.

Skakbrættets funktion er at indsamle data til hovedcomputeren, som bestemmer, hvor UR5-robotten skal flytte hen. Den fungerer også som et tovejs brugerinterface.

Video af brættet funktionalitet(LED'erne er langt mere synlige i virkeligheden):

https://www.youtube.com/shorts/_xA17kUxkV8

5.2 Gripper og skakbrikker



Figur 5: Skakbrik (bonde), blå markering: kontaktpunkt



Figur 6: Gripper åben (15°)



Figur 7: Gripper lukket (-2.3°)

Griberen er designet ud fra et simpelt og pålideligt mekanisk princip: jo færre bevægelige dele, desto lavere kompleksitet og færre potentielle fejlkilder. Derfor er griberen konstrueret med én enkelt bevægelig klo, hvilket reducerer risikoen for mekaniske fejl og forenkler styringen. Der findes diagram tegninger af griberen i figur 20, 21 og 22 i Appendix.

Kloernes geometri er udformet som et halv-oval greb (halvmåneform), dette fremgår tydeligst af figur 7. Når griberen åbnes svarer afstanden mellem indersiden af kloerne til bredden af skakfeltet. Dette design sikrer, at selv hvis brikken ikke er præcist centreret på feltet, vil kloens indadbuede form automatisk skubbe brikken på plads under lukningen. Derved opnås en selvcentrerende effekt, som muliggør konsekvent og stabil gribning, uanset mindre afvigelser i brikkenes placering.

Griberen lukker typisk ned til en vinkel på omkring -2.3° som set i figur 7, hvilket svarer til den nødvendige lukkeposition for at få fat i brikkenes indhak (kontaktpunkt). I åben tilstand står kloen i en vinkel på ca. 15° som set i figur 6, hvilket sikrer tilstrækkelig åbning til at kunne dække hele feltet og gribe omkring brikkerne uden risiko for kollision med nabofelter.

Skakbrikkerne blev udvalgt ud fra et funktionelt krav om, at de skulle indeholde et indhak langs

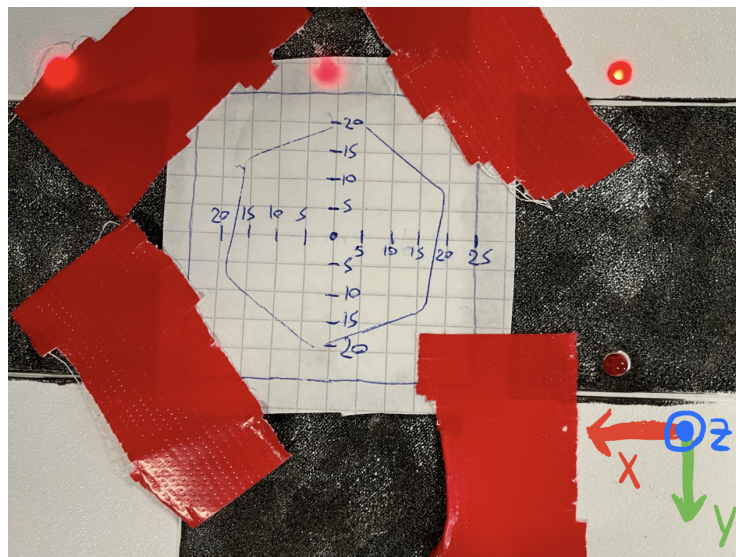
siden, der kunne fungere som griberens primære kontaktpunkt. Brikkerne blev fundet online [4] og er vist i figur 5. For at sikre passende proportioner i forhold til skakfeltets dimensioner blev 3D-modellen skaleret med en faktor 1.5, som dokumenteret i figur 19 i Appendix.

Indhaket er placeret så det flugter med Z-koordinaten i `grid_pick`-matricen, som definerer gribepositionerne i robotkoordinatsystemet (jf. analyseafsnittet om UR5 og ROS).

Brikkernes indhak bidrager desuden med en vigtig mekanisk egenskab: selv hvis griberen rammer brikken lidt over eller under det ideelle kontaktpunkt, vil brikkens geometri samt kloens halvcirkelformede geometri automatisk lede den ned i selve indhaket under lukningen. Dette skaber en selvcentrerende effekt, som forbedrer robustheden af gribeforløbet og reducerer behovet for perfekt positionering.

5.2.1 Testresultater fra gripper og skakbrik sammenspil

For at bestemme gripperens tolerancer og evaluere den selvcentrerende effekt og opsamlings radius blev det undersøgt, hvor langt en skakbrik kunne placeres fra feltets centrum langs X- og Y-aksen og stadig opfanges korrekt af griberen.



Figur 8: Den hexagonale form repræsenterer brikkens base og testpositioner

Som vist i figur 8 blev brikkens centrum forskudt i definerede trin langs både X- og Y-retningen. For hver måleposition blev testen gentaget tre gange. En enkelt fejl i én af gentagelserne blev

betragtet som en total fejl for den pågældende position, da systemet ikke tillader fejlmargen i en aktiv spiltilstand.

Testen blev udført med den højeste brik (dronningen), da dens højde udgør den største risiko for utilsigtet kontakt med griberens roterende arm ved lateral forskydning. Dermed fungerer denne test som en øvre grænse for worst-case-scenarier i forhold til gribersikkerhed og præcision.

Resultaterne er sammenfattet i tabel 1.

Forskydningsretning	Maksimal tilladt afvigelse (mm)
X-aksen	-7.5 / +20
Y-aksen	± 20

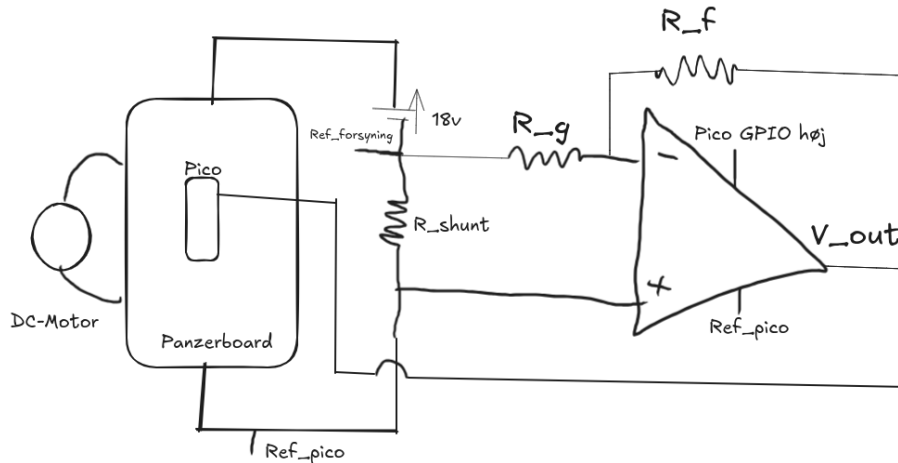
Tabel 1: Maksimale tilladelige forskydninger for vellykket pick-up

5.2.2 Gripper Elektronik

Til aktivering og deaktivering af gripperen anvendtes en DC-motor. Valget af denne motortype var dels bestemt af opgavens krav, men også begrundet i dens enkelhed, lave omkostninger samt det forhold, at applikationen udelukkende krævede rotation i to retninger og mulighed for strømmåling.

Styringen af DC-motoren blev udført ved hjælp af en H-bro integreret på *Panza Boardet*, udviklet af Anders Sørensen. Transistorerne i H-broen blev drevet af et PWM-signal med 20% duty cycle, genereret af en Raspberry Pi Pico. Dette signal regulerede en 18 V strømforsyning til motoren. Anvendelsen af en H-bro muliggjorde både ændring af rotationsretningen og modulation af motorspændingen med PWM fra en lavspændingsmikrocontroller.

For at identificere det tidspunkt, hvor gripperen møder modstand (f.eks. ved kontakt med en skakbrik), blev strømmen gennem motoren målt. Når motoren belastes mekanisk, falder dens interne impedans, hvilket resulterer i en øget strøm. Denne egenskab blev udnyttet til at detektere, hvornår motoren skal stoppes. Målingen af strømmen blev udført ved at registrere spændingsfaldet over en shuntmodstand, som efterfølgende blev forstærket med en ikke-inverterende operationsforstærker. Det resulterende signal blev analyseret med en 12-bit analog-til-digital konverter (ADC) på Pico'en, hvilket gjorde det muligt at følge ændringer i strømmen med høj opløsning og reagere hurtigt på belastningsændringer.



Figur 9: Elektrisk diagram over DC-motor kredsløb

Vi valgte at anvende en forstærkning på 30, idet den målte strøm gennem motoren uden belastning maksimalt var 80 mA. Dette resulterede i et spændingsfald over shuntmodstanden på

$$V_{\text{shunt}} = 0,08 \cdot 0,5 = 0,04 \text{ V},$$

hvor $0,5 \Omega$ er værdien af shuntmodstanden. Med en forstærkning på 30 opnås således et udgangssignal på

$$V_{\text{out}} = 30 \cdot 0,04 = 1,2 \text{ V},$$

når motoren kører uden modstand. Denne forstærkning medfører, at selv små ændringer i strømmen resulterer i tilsvarende større variationer i spænding, hvilket øger opløsningen ved efterfølgende digital konvertering via ADC'en på Pico'en.

Ved analyse af kredsløbet identificeredes, at Pico'ens og strømforsyningens jordreferencer ikke er sammenfaldende, da shuntmodstanden er placeret i serie mellem boardets ground og forsyningens ground. Derfor er der en potentiel forskel i referencepotentiale ved måling af V_{ADC} . Denne forskel svarer til spændingsfaldet over shuntmodstanden, da Pico'en er forbundet til den ene side og strømforsyningen til den anden. Dette kan formelt skrives som:

$$\text{Ref}_{\text{ADC}} - \text{Ref}_{\text{supply}} = v_2 - v_1.$$

Da den ikke-inverterende indgang på operationsforstærkeren er forbundet direkte til v_2 , og den inverterende indgang til v_1 , kan indgangsspændingen til forstærkeren skrives som:

$$V_{\text{in}} = v_2 - v_1.$$

Ved anvendelse af standardformlen for en ikke-inverterende forstærker med reference forskellen trukket fra fås:

$$\begin{aligned} V_{\text{ADC}} &= (v_2 - v_1) \left(1 + \frac{R_f}{R_g} \right) - (v_2 - v_1) \\ &= (v_2 - v_1) + (v_2 - v_1) \frac{R_f}{R_g} - (v_2 - v_1) \\ &= (v_2 - v_1) \frac{R_f}{R_g} \\ &= V_{\text{in}} \frac{R_f}{R_g}. \end{aligned}$$

For at bestemme spændingsfaldet over shuntmodstanden isoleret, divideres V_{ADC} med forstærkningen:

$$V_{\text{shunt}} = \frac{V_{\text{ADC}}}{\frac{R_f}{R_g}} = V_{\text{ADC}} \cdot \frac{R_g}{R_f}.$$

Ved anvendelse af Ohms lov fås strømmen gennem shunten som:

$$I_{\text{shunt}} = \frac{V_{\text{shunt}}}{0,5}.$$

Da shunten er forbundet i serie med Panza-boardets ground, gælder det, at

$$I_{\text{Panza board}} = I_{\text{shunt}}.$$

Boardet forsyner både Pico'en og motoren, og den samlede strøm består derfor af:

$$I_{\text{Panza board}} = I_{\text{motor}} + I_{\text{Pico}}.$$

For at isolere motorens strømforbrug, bestemtes I_{Pico} eksperimentelt. Ved at frakoble motoren ($I_{\text{motor}} = 0$) og måle den resterende strøm til boardet, fandt vi:

$$I_{\text{Panza board}} = 20 \text{ mA} \quad \Rightarrow \quad I_{\text{Pico}} = 0,02 \text{ A}.$$

Motorens strøm kan dermed bestemmes som:

$$I_{\text{motor}} = I_{\text{shunt}} - 0,02 \text{ A}.$$

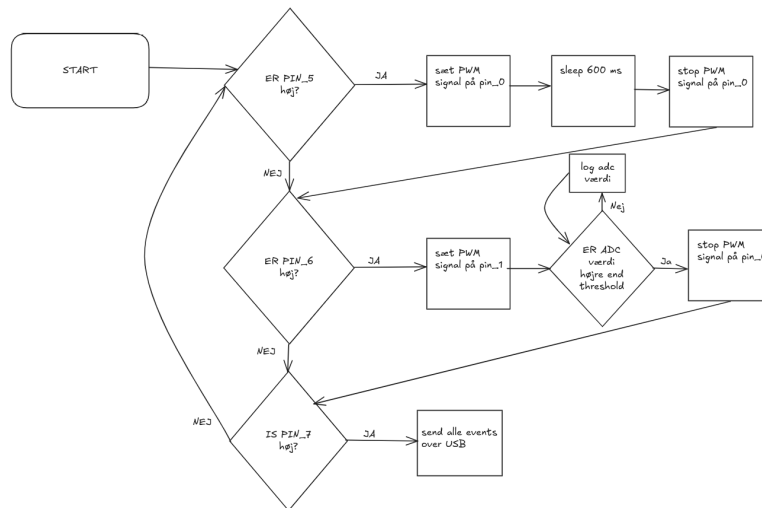
Shuntmodstanden blev placeret i serie med Panza-boardets jord for at holde indgangsspændingen til forstærkeren lav, hvilket muliggør anvendelse af Pico'ens 3,3V udgang som forsyning til MCP6002-forstærkeren. Denne konfiguration beskytter den resterende elektronik, da forstærkerens udgang ikke kan overstige Pico'ens forsyningsspænding.

5.2.3 Gripper Software

Softwaren på gripperens mikrocontroller er struktureret som en state machine med to primære funktioner: `opengripper()` og `closegripper()`, som kontinuerligt afvikles i et uendeligt `while`-loop. Funktionen `opengripper()` aktiverer et PWM-signal i op til 700 ms eller indtil motoren registrerer modstand via øget strømforbrug. Funktionen `closegripper()` aktiverer tilsvarende et PWM-signal i 200 ms, men standser først, når ADC-værdien overstiger en defineret tærskelværdi. Den indledende varighed på 200 ms er nødvendig for at overvinde den statiske friktion, som forårsager et midlertidigt forhøjet strømforbrug i de første ca. 150 ms (se afsnit 5.2.4 for testresultater).

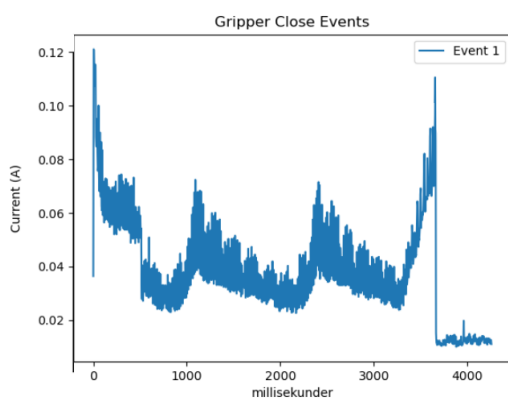
Systemets styringsarkitektur er decentraliseret, idet al beslutningslogik håndteres af hovedcomputeren. Mikrocontrollerens opgave begrænser sig derfor til en simpel state machine, som styrer motorens aktivitet baseret på input modtaget fra hovedcomputeren.

Derudover logger Pico'en alle ADC-målinger i en vektor, hver gang motoren er aktiv. Disse data kan efterfølgende tilgås via UART-forbindelse gennem USB-porten.

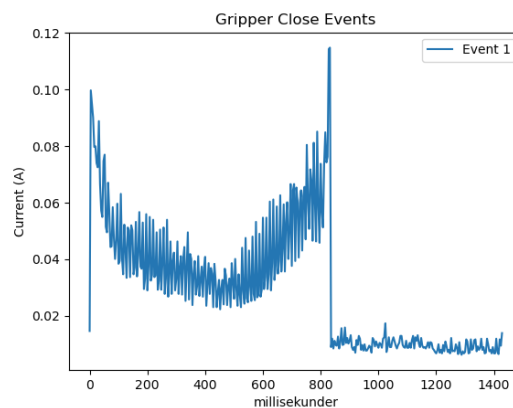


Figur 10: Gripperkode Flowchart

5.2.4 Testresultater fra gripper



(a) 3,5 sekunder måling



(b) Opsamling af en skakbrik

Figur 11: (a) Strøm gennem motoren uden belastning over 3,5 sekunder. (b) Strøमुदvikling ved opsamling af en skakbrik.

Denne test blev gennemført for at undersøge, hvordan strømmen gennem motoren uden belastning ændrer sig over tid (se figur 11(a)). Under forsøget blev der observeret store udsving i strømmen, hvilket antages at skyldes, at selv en lille smule støj i kredsløbet forstærkes betydeligt af operationsforstærkeren. En anden mulig forklaring er, at antagelsen om en konstant strøm fra Pico'en har været upålidelig og derved gjort strøमुdregningerne mindre præcise.

Det primære formål med testen var dog at fastslå den maksimale strøm gennem motoren for at sikre, at gripperen ikke afbryder bevægelsen for tidligt. Denne maksimale værdi blev aflæst til ca.

80 mA på et givet tidspunkt. Som det fremgår af kurven, stiger strømmen markant, når motoren møder belastning. Derfor vurderes variationen i den målte strøm ikke at påvirke det fysiske resultat væsentligt.

I figur 11(b) vises strømforløbet ved løft af en skakbrik. På baggrund af disse målinger er det valgt at stoppe motoren, når strømmen overstiger 100 mA. Det ses desuden, at den statiske friktion medfører en tydelig stigning i strømmen i de første ca. 200 ms efter aktivering. Af denne grund er softwaren programmeret til at ignorere ADC-målinger i dette tidsrum.

5.3 Pico-UR5-Ubuntu Kommunikation

I dette projekt er der anvendt én hovedcomputer, som håndterer al overordnet logik. Derudover benyttes to Raspberry Pi Pico-mikrokontrollere: én til styring af skakbrættets funktionalitet og én til håndtering af gripperen.

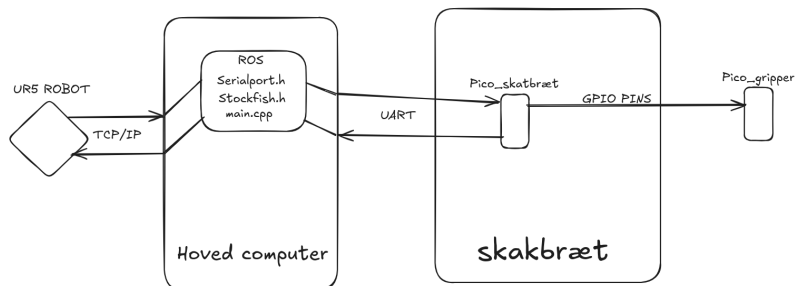
Kommunikationen mellem hovedcomputeren og de enkelte Pico'er foregår via en specialudviklet headerfil, designet specifikt til projektets behov. Denne headerfil indeholder funktioner til både at sende data til Pico'en via UART og til at modtage og læse data, som returneres fra Pico'en gennem samme interface.

Til den Pico, der styrer skakbrættet, er der tilsluttet et USB-kabel, som anvendes til UART-kommunikation samt til forsyning af 5 V. Valget af USB frem for trådløs kommunikation er begrundet i, at robotten er stationær, og at skakbrættets funktionalitet kræver overførsel af lange vektorstreng. En kablet forbindelse sikrer stabil datakommunikation og eliminerer potentielle fejlkilder forbundet med trådløs transmission.

Griperens styring varetages af den anden Pico, som er forbundet til `Pico_skakbræt` via to digitale signalpins samt et fælles ground. `Pico_gripper` overvåger disse signaler for at afgøre, om den skal aktivere `opengripper()` eller `closegripper()`. Signalerne genereres af `Pico_skakbræt`, som modtager instruktioner fra hovedcomputeren via UART. Denne signalformidling håndteres ligeledes gennem den fælles headerfil.

Da kommunikationen mellem de to Pico'er kun kræver to signaltilstande (åben/luk), har vi fravalgt

at implementere UART mellem dem, da dette ville introducere unødigt kompleksitet uden funktionel gevinst.



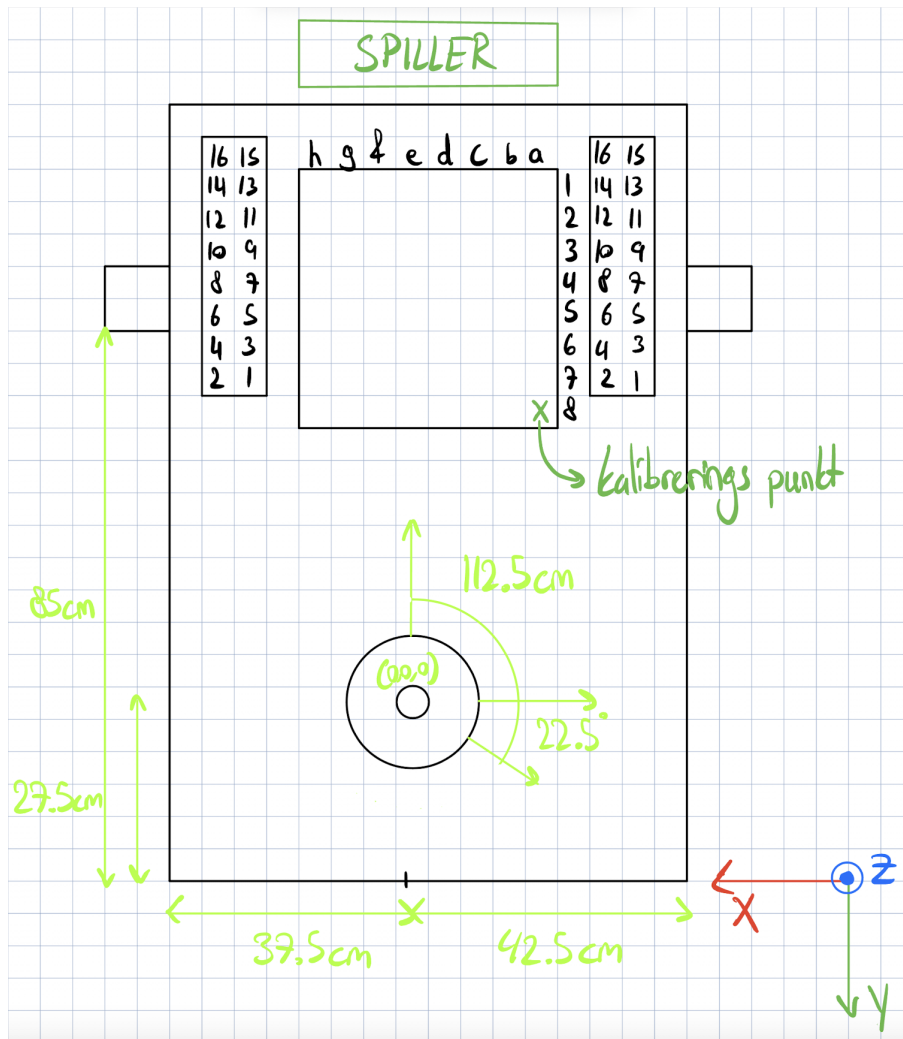
Figur 12: Illustration af systemets forbundne dele

Kommunikationen mellem computeren og UR5 foregår via Ethernet og TCP/IP, hvor ROS-drivers sender bevægelseskommandoer og modtager statusdata via UR's realtidsprotokol RTDE. ROS fungerer som et mellemled mellem brugerens programmering og robotens indbyggede styring.

5.4 UR5 og ROS

ROS blev valgt som styresystem for robotmodellen, da det understøtter integration med visualiseringsværktøjet RViz. Dette muliggør opsætning af robotmiljøet, udvikling af robotkoden, test af kommunikation med Raspberry Pi Pico og verificering af korrekt opsætning uden fare for robotten.

Robotten er monteret med en forskydning på 22.5° som set i figur 13. For at justere robotmodellens retning i forhold til ROS' konventionelle koordinatsystem (REP-103) blev der indført et kunstigt led mellem `base_link` og `shoulder_link` kaldet `base_link.inertia`. Dette mellemled bruges til at introducere en rotationsforskydning på $22,5^\circ$ omkring Z-aksen i det faste led mellem `base_link` og `base_link.inertia`. Dette blev gjort ved at specificere en rpy-rotation i URDF'en svarende til $-22,5^\circ$, hvilket korrigerer for en forskydning i basisrammen og sikrer korrekt overensstemmelse mellem robotmodellens og RViz' visning uden at forstyrre robotens kinematiske egenskaber.



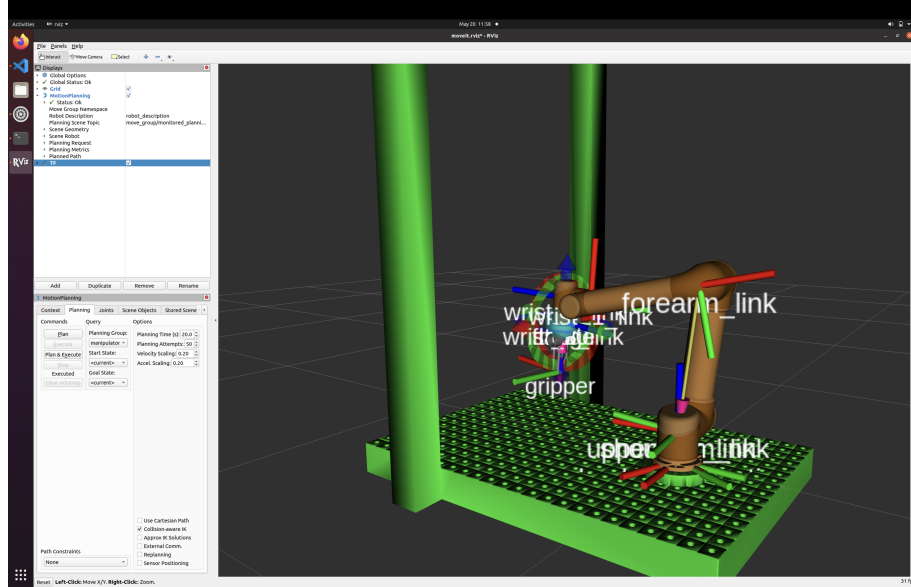
Figur 13: Opsætning

Der blev tilføjet et nyt link med navnet gripper som set i figur 14, som repræsenterer en simpel gripper monteret for enden af robotarmen. Griberen blev implementeret som en STL-mesh med tilhørende visuel og kollisionsgeometri samt inertiparametre. Den er tilknyttet robotten via en fixed joint til tool0, hvilket sikrer, at griberen bevæger sig stift sammen med robotarmens endeffektor. Dette svarer til en forskydning på 16,9 cm frem langs Z-aksen, og en rotation for at justere griberens orientering, så den matcher robotarmens koordinatsystem og griber korrekt nedad.

Kalibreringspunktet som set i figur 13 bruges til at kalibrer gripperens åbne og lukke funktion og sikre korrekt positionering af skakbrættet.

Arbejdsmiljøet, altså metalbordet som set med grøn i figur 14, blev ikke tilføjet via URDF/Xacro-

modellen, det er i stedet tilføjet programmatisk til MoveIt's planning scene ved hjælp af PlanningSceneInterface. Dette gør det muligt at modellere bordet som et kollisionsobjekt i simuleringen, uden at ændre selve robotmodellens struktur.



Figur 14: RViz - Robotmodellen set i "hjemmeposition".

For at håndtere kommunikation mellem Stockfish/Brættet, ROS og robotten blev der udviklet en række hjælpefunktioner, der opererer på 4×8 "skak"matricer og 6×2 opbevaringsmatricer. Disse matricer muliggør præcis positionering, dynamisk og logisk håndtering af skakspillets tilstand. Hjælpefunktionerne forklares i takt med, at de anvendes igennem underkapitlet.

Matricerne vist i figur 15 er en repræsentation af dem, som bruges til "skak"matricerne og sammenlignelig med dem som bruges til opbevaring.

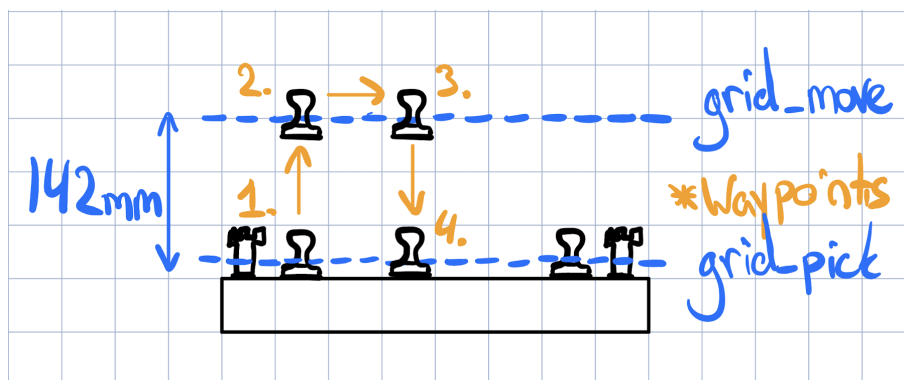
$\begin{bmatrix} r & n & b & q & k & b & n & r \\ p & p & p & p & p & p & p & p \\ & & & & & & & \\ & & & & & & & \\ & & & & & & & \\ & & & & & & & \\ P & P & P & P & P & P & P & P \\ R & N & B & Q & K & B & N & R \end{bmatrix}$	$\begin{bmatrix} A8 & B8 & C8 & D8 & E8 & F8 & G8 & H8 \\ A7 & B7 & C7 & D7 & E7 & F7 & G7 & H7 \\ A6 & B6 & C6 & D6 & E6 & F6 & G6 & H6 \\ A5 & B5 & C5 & D5 & E5 & F5 & G5 & H5 \\ A4 & B4 & C4 & D4 & E4 & F4 & G4 & H4 \\ A3 & B3 & C3 & D3 & E3 & F3 & G3 & H3 \\ A2 & B2 & C2 & D2 & E2 & F2 & G2 & H2 \\ A1 & B1 & C1 & D1 & E1 & F1 & G1 & H1 \end{bmatrix}$	$\begin{bmatrix} pd & pd & pd & pd & pd & pd & pd & pd \\ pd & pd & pd & pd & pd & pd & pd & pd \\ pd & pd & pd & pd & pd & pd & pd & pd \\ pd & pd & pd & pd & pd & pd & pd & pd \\ pd & pd & pd & pd & pd & pd & pd & pd \\ pd & pd & pd & pd & pd & pd & pd & pd \\ pd & pd & pd & pd & pd & pd & pd & pd \\ pd & pd & pd & pd & pd & pd & pd & pd \end{bmatrix}$
Brikkematrixen	Koordinatmatricen	Robotmatricen

Figur 15: Forskellige matricerepræsentationer af et skakbræt

- **Brikkematrixen** gemmer den aktuelle placering af hver skakbrik, fx "p"for bonde eller "k"for konge.
- **Koordinatmatricen** indeholder feltlabels i skaknotation, såsom "e2"eller "g1", og bruges

til at oversætte træk fra Stockfish.

- Der anvendes to **robotmatricer**, kaldet `grid_pick` og `grid_move`, som begge indeholder 3D-positioner (af typen `PoseData`) for hver feltplacering på skakbrættet i robotkoordinater. `grid_pick` indeholder positioner præcis ved brikernes grip placering, mens `grid_move` har identiske X- og Y-koordinater, men med en Z-komponent hævet med 142 mm for at sikre tilstrækkelig frihøjde over den højeste brik som kongen, under horisontale bevægelser og dermed undgå kollisioner med andre brikker.



Figur 16: Visualisering af `grid_pick` og `grid_move` matricer (waypoints)

Matricerne er identisk indekserede, hvilket betyder, at samme indekspar i hver matrice refererer til det samme skakfelt. Et skaktræk som "b7b5" bliver eksempelvis splittet op i "b7" og "b5" vha. funktionen `split("b7b5")`. Ved at søge efter "b7" vha. funktionen `find_in_matrix(koordinatmatricen, "b7")` i koordinatmatricen findes det tilhørende indeks `b7=(row, col)` og `b5=(row2, col2)`, som dernæst bruges til at:

1. Identificere hvilken brik der står på startfeltet og hvilken (hvis nogen) står på målfeltet via brikkematricen.
2. Hvis målfeltet er tomt, springers over til trin 3. Hvis feltet er optaget af en modstanderbrik, flyttes denne først til en opbevaringscontainer (repræsenteret af separate lagermatricer), hvorefter brikken fra startfeltet flyttes til målfeltet.
3. Udføre en bevægelse vha. `move_cartesian_to_pose()`, hvor der planlægges en sti fra punkt 1. i figur 16: `grid_pick[row][col]` og gripperen lukkes, til punkt 2. i figur 16: `grid_move[row][col]`, og videre til punkt 3. i figur 16: `grid_move[row2][col2]` og punkt 4. i figur 16: `grid_pick[row2][col2]`

hvor gripperen åbnes. Gripperen kører derefter op til punkt 4. i figur 16: `grid_pick[row2][col2]`

4. Opdatere brikkematricen vha. funktionen `move_chess_piece_viz()` for at afspejle den nye brikplacering.
5. Robotten returneres til dets "hjemmeposition" vha. `moveToHomePosition()` som eksekverer en bevægelse i joint-space dette er set i figure 14.

`moveToHomePosition()`-funktionen, visualiseret i figur 14, spiller en essentiel rolle i systemets robusthed og sikkerhed. Funktionen sørger for, at robotarmen flyttes til en neutral position uden for skakbrættet efter hvert træk, hvilket skaber et sikkert og forudsigeligt miljø for spilleren. Derudover indføres en bevidst forsinkelse for at give spilleren tid til at flytte sin hånd tilbage efter deres træk uden risiko for kollision.

Da `moveToHomePosition()` opererer i joint-space med faste ledværdier, sikres høj gentagelsesnøjagtighed og deterministisk adfærd, hvilket er afgørende for en sekventiel, turn-baseret applikation som skak.

Den valgte joint-konfiguration i funktionen svarer til **Pattern 6** som set i figur 17 i appendix, karakteriseret ved højre-skulder, albue-op og et ydre håndled (wrist outer). Denne konfiguration blev valgt ud fra både kinematiske og praktiske hensyn, da den:

- minimerer risikoen for kinematiske singulariteter — eksempelvis undgås både albue- og håndledssingulariteter, da armens led holdes i vinkler med høj Jacobian-rang.
- maksimerer robotarmens rækkevidde og fleksibilitet i arbejdsområdet, hvilket er særligt vigtigt i skakmiljøet, hvor robotten ofte skal navigere horisontalt over høje brikker som kongen.

Sammen bidrager disse egenskaber til en stabil og kollisionsfri bevægelsesplanlægning vha.

`move_cartesian_to_pose()` gennem hele spilhåndteringen.

Funktionen `move_cartesian_to_pose()` bruges i projektet til at planlægge og udføre kontinuerlige bevægelser i det kartesiske rum fra robotarmens nuværende position til en ønsket slutposition (PoseData). Denne tilgang sikrer, at robotten bevæger sig i en direkte linje i 3D-rummet.

For at sikre glidende og kontrolleret bevægelse er hastigheds- og accelerationsskaleringsfaktorerne

sat til 0.05, svarende til 5% af robotarmens maksimale kapacitet. Dette valg er truffet for at undgå for hurtige eller ustabile bevægelser, som kan føre til fare for spilleren.

Hvis MoveIt ikke er i stand til at planlægge hele den ønskede bane (mindst 99%), udfører funktionen automatisk et fallback til inverse kinematics, hvor bevægelsen i stedet planlægges i led-rummet. Dette sikrer, at robotten stadig kan nå målet, selv hvis kartesisk planlægning mislykkes — dog uden garanteret retlinet bevægelse.

5.4.1 Testresultater fra UR5

For at undersøge sammenspillet mellem robotarmens bevægelser og griberens åbne-/lukke-funktion blev der udført en test, hvor fokus var at evaluere behovet for forsinkelser mellem bevægelseskommandoer og aktivering af griberen.

Testen bestod af 100 gentagelser af en sekvens, hvor robotten ved hjælp af funktionen `move_cartesian_to_pose()`:

1. bevæger sig vertikalt ned til et opsamlingspunkt,
2. lukker griberen om brikken,
3. bevæger sig op igen, gentager bevægelsen nedad,
4. åbner griberen,
5. og til sidst bevæger sig op igen.

Resultatet af testen viste følgende:

- **2 tilfælde**, hvor griberen først begyndte at åbne sig, da robotten allerede var løftet cirka 1 cm — hvilket indikerer, at der manglede tilstrækkelig ventetid mellem bevægelse og griberaktivering.
- **1 forekomst af en preempted-fejl**, hvor en bevægelse blev afbrudt, sandsynligvis grundet overlap mellem successive bevægelseskommandoer.

Disse observationer og deres implikationer for systemets pålidelighed og timing diskuteres nærmere i diskussionsafsnittet.

6 Diskussion

6.1 Versionskontrol – Git

Vi har forsøgt at udøve versionskontrol gennem Github. På trods af dette lykkedes det dog ikke til fulde, da der hurtigt blev oprettet *branches*, der ikke blev opdateret til/med *main* og derved kom ud af takt. Samtidigt med dette var det først i slutningen af projektets forløb, at koden for kommunikation med og kontrol af UR5 robotten blev lagt op på Git. Dette gjorde det til tider svært for gruppen at få indsigt i og indflydelse på de andre dele af projektet, da man ofte skulle ind på en anden branch eller et andet *repository* for at tilgå den kode, man ville se.

Vi kunne som gruppe have gjort en større indsats for at vedligeholde vores Github repository og dele koden med hinanden, så der var mulighed for bedre samarbejde på tværs af opgaverne.

Vi gjorde også brug af Githubs funktion til projektadministration. Vi tilkoblede et projekt til vores repository med det formål at udnytte *kanban-board* og roadmap til at organisere vores planer. Kanban-boardet blev dog ikke opdateret og revideret regelmæssigt, og blev derfor hurtigt forældet.

Ligesom før kunne vi som gruppe have gjort en større indsats ved for eksempel at holde ugentlige SCRUM-møder, så kanban-boardet blev opdateret regelmæssigt og kunne hjælpe os med at holde overblik over projektet.

6.2 Gripper og Skakbræt

Som det fremgår af afsnittet om testresultater, havde vi store udsving i den målte strøm. Det har dog ikke givet os direkte konsekvenser i forhold til at stoppe motoren i tide. Strømmen stiger nemlig kraftigt, når motoren går fra ingen modstand til næsten fuld modstand på meget kort tid, og denne forskel forstærkes 30 gange. Selvom vores løsning ikke er perfekt for at levere fuldstændigt pålidelige og præcise data, leverer den stadig værdifulde indsigt til strømforbruget.

Vi vil dog sige, at vores endelige resultat var tilfredsstillende. Vi kunne både bruge ADC-målingerne til at lukke gripperen, når det var nødvendigt, og samtidig indsamle data om, hvor meget strøm der løb på et givent tidspunkt.

Hvis man skulle videreudvikle gripperen eller give den flere funktioner, kunne det være en fordel, at hovedcomputeren og gripperen kunne kommunikere direkte. Det ville enten kræve direkte USB mellem `pico_gripper` og hovedcomputeren eller en mere avanceret kommunikation mellem `pico_gripper` og `Pico_skakbræt`. Dette har vi vurderet til ikke at være nødvendigt.

Resultatet for pick-up radiustesten som vist i tabel 1 er fyldestgørende, forklaring på hvorfor brikken kan forskydes 20mm i alle retninger på nær i -x-aksen er grundet den faste arm i gripperen som blot vertikalt vil ramme ned i brikken og forsage fejl.

Dog blev der ikke testet for variationer i Z-retningen, da det oprindeligt blev antaget, at skakbrættet ville være plant.

Under implementeringen viste det sig imidlertid, at skakbrættet har en konkav geometri, forårsaget af utilstrækkelig strukturel understøttelse i midten. Dette har resulteret i, at brættets midte ligger op til 4 mm lavere end kanterne. Dermed opstår der en uventet variation i Z-koordinaten afhængigt af feltets placering.

På trods af dette viste praksis, at griberen stadig formåede at opfange brikkerne korrekt, hvilket i høj grad tilskrives brikernes design med et tydeligt sideindhak som kontaktpunkt og griberens halvcirkelformede klo som passer ind i indhakkets.

Dette bekræfter hypotesen om, at indhakkets på brikken øger tolerancen for højdeafvigelser under pick-up og giver griberen robusthed i forhold til uforudsete geometriændringer i spillemiljøet.

6.3 Op amp

Hvis man gerne ville have undgået at ændre på ground mellem strømforsyningen og Pico'en, kunne man have sat shuntmodstanden i serie med den positive indgang på Panza Boardet. Det ville medføre, at vi arbejdede med højere spændinger og dermed skulle bruge en anden forstærker. Derudover skulle vi også spændingsdele V_{out} for at sikre, at Pico'en aldrig kunne blive ødelagt ved uforventet høj spændingsfald over shunten. Begge løsninger har i sidste ende meget lav effekt på resten af kredsløbet, og derfor har vi valgt at have shuntmodstanden i serie med ground og blot tage højde for forskellen mellem grounds. Desuden ville der alligevel være en forskel i ground, fordi

der er et lille spændingsfald i ledningen fra Panza Boardet til strømforsyningen, som vi har valgt ikke at inkludere i vores beregninger.

6.4 UR5 og ROS

Cartesian-space bevægelse blev valgt frem for joint-space til at håndtere skakbrikkernes bevægelser, da det muliggør præcis baneplanlægning i XYZ-koordinater og sikrer, at griberen bevæger sig i en retlinet og forudsigelig bane. Denne egenskab er alt afgørende, da skakbrikkerne er tæt placeret, og menneskesikkerhed står på spil. Dette står i kontrast til joint-space, hvor kun start- og slutpositionen for robotleddets vinkler tages i betragtning, uden hensyn til den faktisk tilbagelagte bane i rummet som kan medføre ”lasso”bevægelser.

Valget af kartesisk bevægelse øger dog risikoen for at ramme kinematiske singulariteter, hvor visse ledpositioner kan føre til ustabile bevægelser. For at imødegå dette er funktionen `moveToHomePosition()` implementeret. Den placerer robotarmen i en veldefineret og gunstig ledkonfiguration før hvert træk, hvilket sikrer høj reachability og reducerer risikoen for singularitet under de efterfølgende kartesiske bevægelser.

Testresultaterne fra sammenspillet mellem robotarmens bevægelser og griberens åbne-/lukke-funktion indikerer, at der – trods sekventiel eksekvering i koden – er behov for eksplicitte forsinkelser mellem robotten og griberen. Dette skyldes formodentlig de eksekveres parallelt, selvom kommandoerne sendes i rækkefølge.

De to observerede tilfælde, hvor robotarmen begyndte at løfte sig, inden griberen var helt åben, kan med høj sandsynlighed forklares af manglende forsinkelser, men også af måden hvorpå `MoveIt` håndterer bevægelseskommandoer. Her bliver nye mål tilføjet til en planlægningskø for hurtigt, hvilket kan resultere i, at robotten ”springer videre” til næste sekvens af bevægelsesbanen, før griberen er færdig med sin handling.

Den ene registrerede `preempted`-fejl vurderes at skyldes enten tidsmæssigt overlap mellem nye bevægelseskommandoer eller en timeout i bevægelsesplanlægningen. Da fejlen kun opstod én gang ud af 100 gentagelser og ikke blev observeret under faktisk spil, efter nødvendige forsinkelser blev

indført, er den ikke undersøgt yderligere.

Introduktionen af yderligere forsinkelser har minimeret disse hændelser men ikke komplet elimineret dem i spillestanden.

6.5 Skakbræt eller Computer Vision

Da vi besluttede at lave skak, skulle vi finde en metode til at aflæse spillerens træk. Den metode, der typisk bruges i, er computer vision, hvor et kamera tager et billede af brættet, og en computer analyserer alle pixels for at se, hvad der har ændret sig fra billede til billede. Men en af de ting, vi fandt spændende, var et system, hvor selve brættet kunne lyse de felter op, hvor spilleren må rykke, samt andre animationer. Med disse ambitioner ville computer vision være næsten umuligt, da spillerens hånd ville blokere for kameraet, når brikken løftes. Derudover skulle vi alligevel lave elektronik under brættet til LED'erne, så vi valgte i stedet at bruge magnetisme til at aflæse positioner.

Vores løsning endte med at være tilfredsstillende og kunne både modtage de korrekte data fra hovedcomputeren i form af, hvilke lovlige træk spilleren må tage, og sende de korrekte træk, som spilleren har foretaget. Der har dog stadig været nogle problemer med magneterne: Nogle gange kan magneters magnetfelter forstyrre de nabolægende reed-relæer, hvilket betyder, at et reed-relæ, som var tændt, nu er slukket. Vi har for det meste omgået problemet med software, der tjekker, om et relæ er i en ulogisk tilstand. Hvis vi ville undgå problemet fuldstændigt, kunne vi også have brugt selve skakbrikken som en switch, hvor strømmen skal løbe igennem brikken for at aflæse en position.

6.6 Pico-UR5-Ubuntu Kommunikation

Under sammenspillet mellem ROS, MoveIt og Raspberry Pi Pico observeredes der lejlighedsvist tab af forbindelse til Pico'en via serial port (`/dev/ttyACM0`). Disse forbindelsestab er ikke forekommet under individuel test af Pico'en alene eller under isoleret ROS-afvikling, men opstår udelukkende, når flere systemer kører parallelt, herunder MoveIt's planlægningsalgoritmer og robotvisualisering i RViz.

Vi mener fejlen tåntænkes høj systembelastning af den virtuel maskine (VM), hvor håndteringen af USB-enheder, særligt virtuelle serialporte, kan være ustabil ved samtidige processer og IO-aktiviteter. Når systemet belastes, kan Pico'en midlertidigt frakobles.

6.7 Specielle skaktræk

I skak findes der også enkelte specialtræk, som går ud over funktionaliteten af de sædvanlige træk. Disse træk inkluderer: En passant, rokering, samt promovering af bonde. Vi har i vores software taget højde for specialtrækkende på følgende måde:

- Rookering: Rookering har notation som et hvert andet træk i UCI (Universal Chess Interface) med den ene forskel, at kongen foretager en bevægelse på 2 felter. Da kongen normalt kun kan bevæge sig 1 felt ad gangen, kan vi lede efter disse få tilfælde til at markere en rokering.
- Promovering af bonde: En promovering af en bonde tager sted, når bonden når den modsatte spillers bagerste rang (altså rang 8 for hvid, rang 1 for sort). I UCI noteres en promovering som et normalt træk, men med den brik, man promoverer til, tilføjet bagpå. For eksempel en promovering til dronning for hvid ser således ud: a7a8q. Vi leder derfor i vores software efter hvornår brugeren rykker en bonde til sidste rang, og spørger dem så i softwaren hvad de promoverer til.
- En passant: Da dette er en edgecase, som kun kan foretages i meget specielle sammenhæng, har det ikke været vores prioritet, at inkludere dette specialtræk. Dette træk kan derfor hverken foretages af robotten eller spilleren korrekt.

Hvis vi havde prioriteret et færdigt produkt, der kunne foretage alle specialtræk, kunne vi have udviklet et produkt, som også kunne være mere tilrettet professionelle skakspillere. Vi har dog valgt, at gå ud fra, at vores brugere bruger systemet til at lære de basale træk i skak, og derfor ikke kender til alle specialtrækkende.

7 Konklusion

I dette semesterprojekt har vi med succes designet og implementeret et autonomt robotsystem, hvor en UR5-robotarm via en specialudviklet gripper og et elektronisk skakbræt kan spille skak mod en menneskelig spiller. Projektet havde til formål at skabe et fysisk, intuitivt og engagerende skakmiljø uden behov for unødigt skærminteraktion, hvilket blev realiseret gennem brug af magnetiske reed-relæer og visuel feedback med LED'er.

Gripperen blev udviklet med fokus på enkel mekanisk konstruktion, lav strømforbrug og præcis kontrol. Gennem integration af PWM-styret DC-motor og strømmåling via shuntmodstand og operationsforstærker, kunne vi både styre og overvåge gripperens funktionalitet. Testresultater viste, at gripperen pålideligt kunne opsamle skakbrikker, og at strømmålinger effektivt kunne bruges til at bestemme, hvornår gripperen lukkede korrekt.

UR5-robotarmen blev styret via ROS, hvor vi udnyttede både MoveIt og RViz til planlægning og visualisering. Vores softwarearkitektur kombinerede ROS, C++ og Stockfish, hvilket muliggjorde intelligent trækberregning, realtidskommunikation og præcis bevægelsesplanlægning. Systemet formåede at udføre skaktræk sekventielt, præcist og brugervenligt dog med tilfældige kommunikations problemer.

Undervejs i projektet blev der truffet bevidste designvalg, såsom at fravælge computer vision til fordel for en magnetisk registreringsløsning, hvilket viste sig at være en robust og responsiv tilgang. Udfordringer med versionskontrol og Git-samarbejde blev identificeret som et område med forbedringspotentiale, som kunne have styrket den interne kommunikation og vidensdeling yderligere.

Overordnet vurderer vi, at projektets målsætninger er blevet indfriet, og at systemet fungerer tilfredsstillende som en fysisk interaktiv skakrobot. Der er stadig plads til videreudvikling, især i forhold til håndtering af specialtræk som "en passant" og forbedring af datapræcision i gripperens målinger, men vi anser den nuværende løsning som et solidt fundament for et brugervenligt og funktionelt skaktræningsværktøj.

8 Perspektivering

Projektet har demonstreret, hvordan et relativt komplekst samspil mellem software, elektronik, mekanik og spilintelligens kan realiseres i et brugervenligt og funktionelt produkt. Det udviklede system fungerer som et bevis på, at robotteknologi ikke nødvendigvis behøver at være utilgængelig eller kun relevant i industrielle sammenhænge – det kan også skabe værdi i undervisning, læring og leg.

I fremtiden kan projektet videreudvikles på flere fronter. Teknisk kunne systemets præcision og robusthed forbedres ved f.eks. at benytte sensorer eller tilføje taktil feedback til gripperen, så den kan registrere om en brik faktisk er samlet op. Ligeledes kunne softwaren udvides til at håndtere samtlige specialtræk i skak samt flere spilvarianter, hvilket ville gøre systemet mere anvendeligt for øvede spillere og undervisningssituationer. Et andet oplagt udviklingsspor er brugeroplevelsen. Et interface, hvor spilleren får løbende feedback, eller mulighed for at kommunikere med robotten gennem stemme eller touchscreen, kunne bringe systemet tættere på et færdigt produkt.

Teknologien bag dette projekt kan også overføres til andre formål. Den fysiske manipulation via robotarm og gripper, kombineret med sensor-feedback og intelligent styring, kan tilpasses mange interaktive scenarier – som fx logistiksystemer, undervisningsværktøjer eller træningsudstyr.

På uddannelsesniveau viser projektet værdien af tværfagligt samarbejde og anvendelsesorienteret læring. Det kombinerer teorier fra elektronik, programmering og robotik i en samlet løsning – og illustrerer, hvordan praktisk erfaring kan give dybere forståelse for tekniske problemstillinger og deres løsninger.

Afslutningsvis har projektet givet os en praktisk og tværfaglig forståelse for robottekniske systemer, og det har vist, hvordan samarbejde mellem hardware og software er afgørende for at skabe et sammenhængende, funktionelt og brugervenligt produkt. Med videreudvikling og skalering kunne et sådant system potentielt finde anvendelse både kommercielt og pædagogisk.

9 Litteratur

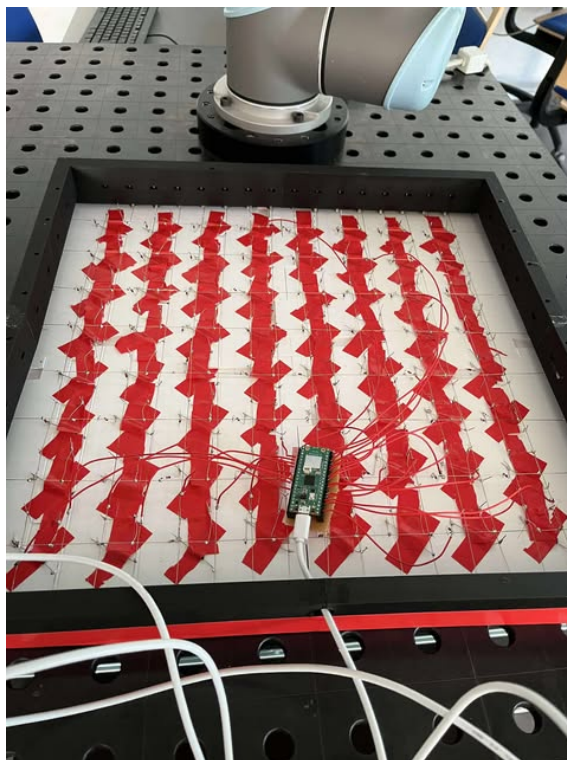
- [1] Thomas Bräunl. *Embedded Robotics*. Accessed: 12/05-2025. URL: <https://link-springer-com.proxy1-bib.sdu.dk/book/10.1007/978-981-16-0804-9>.
- [2] Paul Evans. *DC Motor Explained*. Accessed: 12/05-2025. URL: <https://theengineeringmindset.com/dc-motor-explained/>.
- [3] Iñigo Iturrate. *Lecture 7: Inverse Kinematics*. Kursusmateriale fra SDU, adgang via itsLearning. Accessed: 19/05-2025.
- [4] Matt Kerekanich. *Skakbrikker*. Accessed: 15/05-2025. URL: <https://www.myminifactory.com/object/3d-print-low-poly-chess-set-29582>.
- [5] Universal Robots. *UR5*. Accessed: 19/05-2025. URL: <https://www.universal-robots.com/products/cb3/>.
- [6] Universal Robots. *UR5_ROS*. Accessed: 19/05-2025. URL: https://github.com/UniversalRobots/Universal_Robots_ROS_Driver.
- [7] Open Source. *ROS Wiki*. Accessed: 19/05-2025. URL: <https://wiki.ros.org>.
- [8] Stockfish Dev. Team. *Stockfish FAQ*. Accessed: 12/05-2025. URL: <https://official-stockfish.github.io/docs/stockfish-wiki/Stockfish-FAQ.html>.
- [9] Unknown. *Operational amplifier*. Accessed: 12/05-2025. URL: https://en.wikipedia.org/wiki/Operational_amplifier.
- [10] Unknown. *Shunt (electrical)*. Accessed: 13/05-2025. URL: https://en.wikipedia.org/wiki/Shunt_%28electrical%29.
- [11] Unknown. *Multiplexing*. Accessed: 13/05-2025. URL: <https://en.wikipedia.org/wiki/Multiplexing>.

10 Appendix

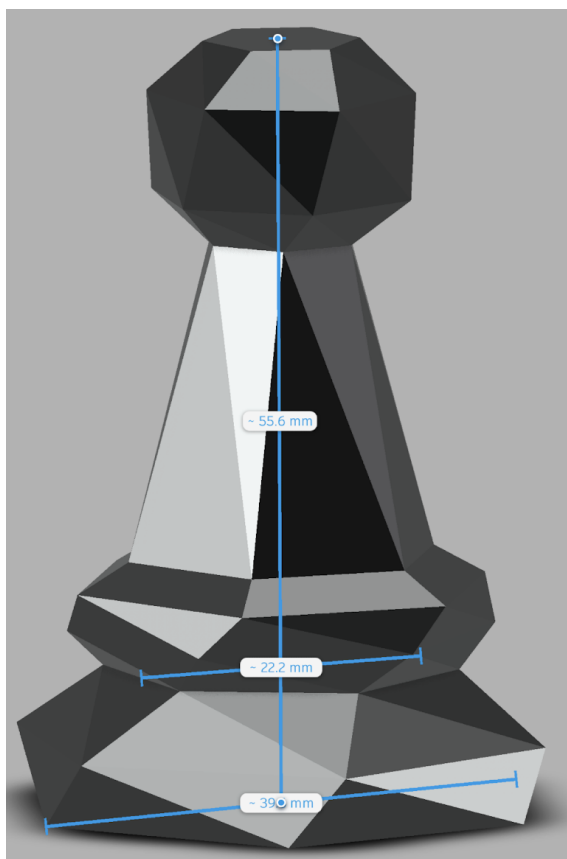
Example – Multiple IK Solutions: UR (6-DOF)									
Pattern	Shoulder	Elbow	Wrist	Figure1 (Tool Down)	Pattern	Shoulder	Elbow	Wrist	Figure1 (Tool Down)
1	Left Side	Down	Tool Down: Outer		5	Right Side	UP	Tool Down: Inner	
2	Left Side	Down	Tool Down: Inner		6	Right Side	UP	Tool Down: Outer	
3	Left Side	UP	Tool Down: Outer		7	Right Side	Down	Tool Down: Inner	
4	Left Side	UP	Tool Down: Inner		8	Right Side	Down	Tool Down: Outer	

SDU  38

Figur 17: Eksempel på inverse kinematiske løsninger for UR-robot (6-DOF). Figuren er hentet fra pensum [kilde: [3]].



Figur 18: Undersiden på det virkelige skakbræt



Figur 19: Bondens dimensioner

